2026.03.02

김태훈

# Advanced RAG Part 2

Query Translation: RAG-Fusion

# 목차


01 **프로젝트 설명**

기존 프로젝트: Multi-Query를 활용한 질문 재생성 → RAG 호출
현 프로젝트 방향성: RAG-Fusion을 활용한 질문 우선순위 지정 및 융합 → RAG 호출

02 **아키텍처 및 작동 원리**

워크 플로우1: Multi-Query 생성
워크플로우 2: 병렬 RAG 실행
워크플로우 3: 중복 Document에 대한 점수 부여 + 중복 제거 (Unique Union)
워크플로우 4: 최종 답변 생성

03 **구현 방법**

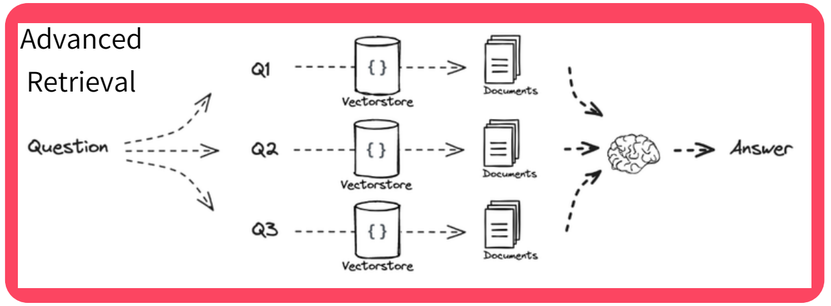
환경 설정: VS Code와 Jupyter Extension을 활용한 개발 환경
코드 구현: Python 기반의 단계별 라이브러리 호출 빛 프로토타입 소스 코드 가이드

04 **Next RAG**
**Decomposition**

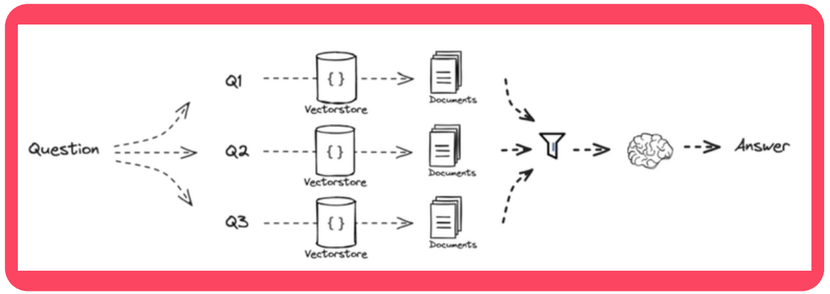
Query Translation 개요 및 추후 아이디어

# 01 프로젝트 설명

## AS-IS: Query Translation: Multi-Query




## TO-BE: Query Translation: RAG-Fusion




RRF(Reciprocal Rank Fusion) 알고리즘: "상호간의 순위 융합"

- 다양한 검색 결과의 순위를 종합하여 검색 순위를 다시 매기는 하이브리드 알고리즘.

# 02 아키텍처 및 작동 원리

### 워크 플로우1: Multi-Query 생성

사용자 질문 1개
↓
LLM이 동일한 질문을 4가지 다른 관점으로 재작성
↓
["질문 버전1", "질문 버전2", "질문 버전3", "질문 버전4"]

### 워크 플로우2: 병렬 RAG 실행

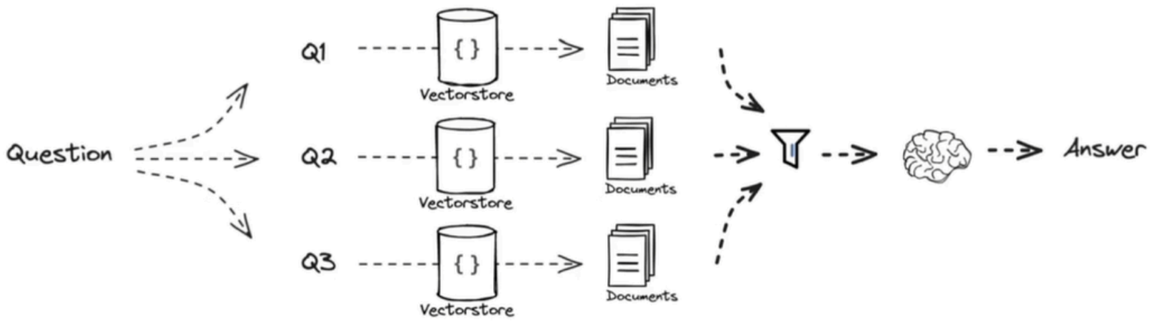
5개의 질문 각각에 대해 retriever를 병렬로 실행 (retriever.map())
↓
[[doc1, doc2, ...], [doc3, doc4, ...], [doc1, doc5, ...], ...]
총 4개의 문서 리스트 반환 (각 리스트당 k개 문서)

### 워크 플로우 3: 순위 등록 + 중복 제거

RRF 점수 계산: 각 문서에 1 / (rank + k) 점수를 누적 합산 → 여러 쿼리에서 반복 등장할수록, 높은 순위일수록 점수 상승
↓
재순위화 및 반환 중복 문서 제거 후 누적 점수 기준 내림차순 정렬 → 최종 통합 문서 리스트 반환

### 워크 플로우4: 최종 답변 생성

unique 문서들을 context로 사용
↓
LLM이 context + 원본 질문을 바탕으로 최종 답변 생성



# 03 코드 구현

https://github.com/KimHoon98/RAG-Implementation

0.) README를 통해  페이지에 대한 설명, 프로젝트 순서를 확인할 수 있습니다.
1.) 각 파일마다 README 파일이 첨부 되어있으니 참조하시면 됩니다.
2.) 해당 파워포인트에 대한 설명은 Note explanation Section 참조하세요.

# 04 Next RAG: Decomposition

## Query Translation

1. Multi-Query: 유저의 질문을 n개의 같은 의미의 다른 형태로 나타내어 커버리지 확대
2. RAG-Fusion: n개의 질문에 대해 받아온 context를 순위로 나누어 LLM에게 전달
3. Decomposition: 구글의 Least-to-most 아이디어: IR-CoT활용